

Workshop 3: Classification models

TECH6

Richard Foltyn

NHH Norwegian School of Economics

October 29, 2026

See GitHub repository for notebooks and data:

<https://github.com/richardfoltyn/TECH6-H26>

Exercise 1: Classification of above-average houses

In this exercise, we use classification to predict whether a house was sold for more than the average price in its neighborhood.

We apply the same preprocessing steps as in other exercises using the Ames housing dataset, which are repeated below for convenience.

```
[1]: import pandas as pd
import numpy as np

# Use this path to use the CSV file from the data/ directory
file = '../data/ames_houses.csv'

df = pd.read_csv(file, sep=',')

# Drop rows with missing observations
df = df.dropna()

# Drop observations with large living or lot area
df = df.query('LivingArea <= 3500 & LotArea <= 5000')

# Create indicator variable for single-family homes
df['IsSingleFamily'] = (df['BuildingType'] == 'Single-family').astype(int)

# Create indicator variable for central air
df['CentralAir'] = df['CentralAir'].map({'Y': 1, 'N': 0})

print(f'Number of observations: {df.shape[0]:,d}')
```

Number of observations: 2,755

Part 1 — Data preprocessing

Perform the following additional data preprocessing steps:

1. Drop all neighborhoods with fewer than 40 observations.
2. Create a new variable `MoreExpensive` that is 1 whenever the sale price is above the average sale price in the neighborhood.

3. Split the dataset into two DataFrames, `df_train` and `df_test`, where the test sample should contain 20% of the observations. Stratify the train-test split using the indicator `MoreExpensive`.

Part 2 — Logistic regression

Using the template code below, create the feature matrix for logistic regression as follows:

1. Create polynomials of degree 3 using the variables `LivingArea`, `LotArea`, `OverallQuality`, `OverallCondition`, `Bathrooms`, `Bedrooms`, `Fireplaces`, and `YearRemodeled`.
2. Add the non-interacted features `CentralAir` and `IsSingleFamily` to the feature matrix.

Then perform the following steps to fit and evaluate the model:

1. Fit a logistic regression with `LogisticRegression()` using the indicator `MoreExpensive` as the target variable.
 - Does logistic regression require feature standardization? If so, you need to transform the features using `StandardScaler()`.
 - You can use the default parameters for `LogisticRegression`, but you might need to increase the maximum number of iterations (e.g., `max_iter=10_000`).
2. After you have fitted the model, use the function `tabulate_classifier_metrics()` defined below to tabulate the accuracy, precision, recall, and F1 score on the test sample.
3. After you have fitted the model, use the function `plot_confusion_matrix()` defined below to plot the confusion matrix on the test sample.

This function calls `ConfusionMatrixDisplay.from_estimator()` to create a confusion matrix graph.

```
[5]: from sklearn.preprocessing import PolynomialFeatures

# Target variable name
target = 'MoreExpensive'

# Features included as polynomials (in logistic regression)
features_poly = [
    'LivingArea',
    'LotArea',
    'OverallQuality',
    'OverallCondition',
    'Bathrooms',
    'Bedrooms',
    'Fireplaces',
    'YearRemodeled',
]

# Other features not included in polynomials
features_other = ['CentralAir', 'IsSingleFamily']
features = features_poly + features_other

# Response variable
y_train = df_train[target]
y_test = df_test[target]

# TODO: Create polynomial features for training sample

# TODO: Create polynomial features for test sample

# TODO: Merge polynomial features and non-polynomial features into X_train
```

```

# TODO: Merge polynomial features and non-polynomial features into X_test

# TODO: Standardize features

# TODO: Fit logistic regression model

# TODO: Tabulate metrics on test sample using tabulate_classifier_metrics()

# TODO: Plot confusion matrix using plot_confusion_matrix()

```

```
[6]: from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score
```

```

def tabulate_classifier_metrics(estimator, X, y):
    """
    Tabulate classification metrics (accuracy, precision, recall, F1).

    Parameters
    -----
    estimator : object
        Fitted classifier.
    X : array-like
        Feature matrix.
    y : array-like
        Target variable.
    """

    # Predict outcome
    y_pred = estimator.predict(X)

    # Compute scores
    acc = accuracy_score(y, y_pred)
    pre = precision_score(y, y_pred)
    rec = recall_score(y, y_pred)
    f1 = f1_score(y, y_pred)

    # Combine scores into a single Series
    index = pd.Index(
        ['Accuracy', 'Precision [TP/(TP+FP)]', 'Recall [TP/P]', 'F1'], name='Metric'
    )
    stats = pd.Series([acc, pre, rec, f1], index=index)

    stats = stats.round(3)

    return stats

```

```
[7]: from sklearn.metrics import ConfusionMatrixDisplay
```

```

def plot_confusion_matrix(estimator, X, y, title='Confusion matrix'):
    """
    Plot confusion matrix for classification model.

    Parameters
    -----
    estimator : estimator
        Fitted classification model.
    X : array-like
        Feature matrix.
    y : array-like
        Target variable.
    title : str

```

```

""" Title of the plot.
"""

cm = ConfusionMatrixDisplay.from_estimator(
    estimator=estimator,
    X=X,
    y=y,
    values_format=',d',
    cmap='Blues',
    colorbar=False,
    text_kw={'fontsize': 10, 'fontweight': 'bold'},
)
cm.ax_.set_title(title)

```

Part 3 — Logistic regression CV

Instead of using the default regularization strength $C=1$, perform cross-validation to find the optimal value of C :

1. Run the cross-validation with `LogisticRegressionCV()`.
Create a log-spaced grid of candidate values as follows:
`C_grid = np.logspace(-2, 2, 500)`
2. Report the optimal value of C .
3. After you have fitted the model, use the function `tabulate_classifier_metrics()` to tabulate the accuracy, precision, recall, and F1 score on the test sample.
4. After you have fitted the model, use the function `plot_confusion_matrix()` defined above to plot the confusion matrix on the test sample.

Part 4 — Random forest

You now want to investigate how other classifiers perform on this task compared to logistic regression.

1. Fit the random forest classifier implemented in `RandomForestClassifier()` to the data. Use the default parameters for now.
 - Do you need to include polynomial interactions for a random forest?
 - Do you need to standardize features for a random forest?
2. After you have fitted the model, use the function `tabulate_classifier_metrics()` to tabulate the accuracy, precision, recall, and F1 score on the test sample.
3. After you have fitted the model, use the function `plot_confusion_matrix()` defined above to plot the confusion matrix on the test sample.

Part 5 — Random forest CV

In the previous part, you used the default hyperparameters for the random forest (e.g., the number of trees to grow and the maximum depth).

1. Perform cross-validation of these parameters with `GridSearchCV()` using the parameter grid defined in the template below.
2. After you have fitted the model, use the function `tabulate_classifier_metrics()` to tabulate the accuracy, precision, recall, and F1 score on the test sample.

3. After you have fitted the model, use the function `plot_confusion_matrix()` defined above to plot the confusion matrix on the test sample.

```
[19]: from sklearn.model_selection import GridSearchCV

# Define hyperparameter grid
param_grid = {
    'n_estimators': np.arange(100, 201, 10),
    'max_depth': np.arange(3, 20),
}

# TODO: Call GridSearchCV to find optimal hyperparameters

# TODO: Report optimal number of estimators stored in best_params_

# TODO: Report optimal max depth stored in best_params_
```

Part 6 — Compare estimation results

Combine the accuracy, precision, recall, and F1 metrics for all the models you estimated and report them in a single table. Which estimator performs best on the classification task?